Secrets Management for the Workflow Construct

Ronald G. Thomas

2026-05-17



Figure 1: A small leather satchel with a brass buckle, open just enough to reveal several distinct small keys and a folded paper ticket inside.

Secrets are state. They deserve the same inventory, backup, and recovery discipline as any other state on the machine.

Introduction

A research workstation accumulates secrets without anyone deciding that it should. An API key for a language model is exported in a shell startup file so that one script will run; an R environment file gains a second key for a package that wraps another service; a cloud provider's access key sits in `~/.aws`; the GitHub command-line tool quietly stores a login token; a backup tool keeps OAuth tokens for three cloud drives. Each was placed in good faith, for one task, and together they form an unrecorded inventory with no rotation schedule and no plan for what happens when one of them leaks.

> **Note**
>
> Part 3 of the security series; start with Security Foundations for a Multi-Machine Research Fleet for the overview.

This post covers the secrets that are not personal passwords: API keys, cloud credentials, and the OAuth tokens that command-line tools hold on our behalf. Personal passwords and the GPG key that protects them are the subject of Setting Up pass. The material here comes from running a five-machine fleet (three Macs, a Linux laptop, and a Linux home server) and from one concrete incident: settings folders that had been symlinked into a cloud sync folder for months, which put several live credentials somewhere they should never have been. That incident, which this series calls the exposure, is told in full in Migrating Off Dropbox. Here it serves as the worked example for rotation.

Motivations

- Credentials leak through channels nobody designed as channels: a synced settings folder, a diagnostic command that prints the environment, a tool that writes its token into a directory that happens to be tracked by git.
- A credential that has leaked must be replaced everywhere it is used. Without an inventory, nobody knows where that is.
- Several tools in daily use (the GitHub CLI, the backup tool `rclone`, editor plugins) hold OAuth tokens that the operator never sees or types, and those tokens are as powerful as passwords.
- Revocation behaves differently from what intuition suggests: an old key can still be live, and revoking one machine's grant can log out every machine.

Objectives

1. A placement rule for every class of secret: where it may live, and where it must never live.
2. A small, tested pattern for loading a secret from `pass` into one command's environment rather than the whole shell session.
3. A practical answer to the R `.Renviron` question, where a tool insists on reading secrets from a file at startup.
4. A rotation procedure that holds up under a real exposure, including the checks that confirm an old credential is actually dead.
5. Two recommended practices for the residual risk: short-lived cloud credentials and pre-commit secret scanning.



Figure 2: A single brass padlock, open, resting beside a small torn paper ticket with a short hand-written code.

What Counts as a Secret, and Where It May Live

The useful unit of analysis is not the individual secret but its class, because each class has a natural home and a characteristic way of leaking.

Table 1: Secret classes and their placement.

Class	Examples	Home	Must never be in
Personal passwords	web accounts, app passwords	`pass`	browser exports, notes
API keys	language-model APIs, data APIs	`pass`, loaded on demand	shell startup files, notebooks
Cloud credentials	AWS access	short-lived tokens; `~/.aws` only if unavoidable	synced folders, dotfiles
Tool OAuth tokens	GitHub CLI, `rclone`	the tool's own store (macOS Keychain where supported)	dotfiles, logs

Class	Examples	Home	Must never be in
SSH private keys	per-relationship keys	`~/.ssh`, one key per relationship	any synced folder
Backup passphrases	Borg repositories	`pass` (and nowhere on the storage host)	the machine holding the repository

Three rules sit underneath the table and apply to every row.

- **Secrets never live in a synced folder or in a dotfiles repository.** Sync clients and git remotes copy files to places with different access controls, retention, and audit trails. A secret that reaches either must be treated as exposed.
- **Load on demand, do not export globally.** A secret exported in `~/.zshrc` is present in every process the shell starts for the rest of the session, including the ones that print their environment.
- **Never print a secret, and rotate any secret that has been printed.** Terminal scrollback, shell history, screen sharing, and session transcripts all retain output.

SSH key policy (one key per relationship, restricted keys enforced on the server) is part of the control set described in the series overview; backup passphrases are covered in Research Backup Architecture. The rest of this post concerns the API keys, cloud credentials, and tool tokens.

Prerequisites

- A working `pass` store with a GPG key and a private git remote, as set up in Setting Up pass.
- Full-disk encryption on every machine that holds a secret, as described in the series overview. A cached GPG passphrase or a local credential file is only as safe as the disk it sits on.
- A dotfiles repository with a `.gitignore` that already excludes secret-bearing paths (shown below).
- About an hour to inventory existing secrets and move them.

Loading Secrets on Demand

The `with-secret` Pattern

The shell helper below reads one entry from `pass` and passes it to a single command as an environment variable. The value never enters the interactive shell's environment and never appears on a command line, so it is absent from `ps` output and shell history.

```
# Run one command with one secret in its environment.
# Usage: with-secret VAR_NAME pass/entry command [args...]
with-secret() {
  local var_name="$1"
  local entry="$2"
  shift 2
  local value
  value="$(pass show "$entry" | head -n 1)" || {
    print -u2 "with-secret: pass lookup failed for '$entry'"
    return 1
  }
  env "$var_name=$value" "$@"
}
```

Taking only the first line follows the `pass` convention that the first line of an entry is the secret and later lines hold metadata. Wrappers for frequently used tools then become one line each:

```
llm-cli() {
  with-secret OPENAI_API_KEY api/openai command llm-cli "$@"
}
```

The contrast with the common alternative is the whole point:

```
# Avoid: visible to every subprocess for the rest of the session
export OPENAI_API_KEY="$(pass show api/openai)"

# Prefer: visible only to the wrapped command and its children
with-secret OPENAI_API_KEY api/openai llm-cli
```

The entry names shown are generic examples; any hierarchy works, and a consistent prefix such as `api/` makes the inventory easy to list with `pass ls api`.

The R `.Renviron` Question

R reads `~/.Renviron` once, at startup, and many packages expect API keys to arrive that way. That design does not fit on-demand loading, and there are two defensible answers.

The first is to keep a small `~/.Renviron` that is local to each machine and treated as a secret file in its own right:

- permissions `600`, so only the owner can read it;
- excluded from the dotfiles repository and from every synced folder;
- included in the machine's encrypted nightly backup, so that losing the machine does not mean losing the keys.

This is the pragmatic arrangement on our fleet for R and for a similar `~/.env` file read by a few scripts. It is safe only if all three conditions hold. The backup condition was added after the quarterly audit found that these files, along with `~/.aws`, were in no backup at all, which is described in Research Backup Architecture.

The second answer is to set the variable for the current session from `pass`, which keeps the key out of any plaintext file:

```
# Set one key for this R session only; nothing is written to disk.
Sys.setenv(
  OPENAI_API_KEY = system2("pass", c("show", "api/openai"),
                           stdout = TRUE)[1]
)
```

This works wherever `gpg-agent` can prompt or already holds the passphrase. It is the better choice for keys that are used rarely, and for any machine where a plaintext file would be difficult to keep out of sync or backup scopes. The snippet is illustrative; it has been checked interactively but is not wired into any startup file on the fleet.

The dotfiles repository protects both arrangements with ignore rules written before the first commit, because removing a committed secret means rewriting history and rotating the secret anyway:

```
**/.env
**/.env.local
**/.env.*.local
**/Renviron
.password-store
.gnupg
.ssh
.aws
```

Passphrase Caching

Each `pass show` asks `gpg-agent` to decrypt, and the agent prompts for the GPG passphrase unless it is cached. A moderate cache lifetime keeps the prompts tolerable without leaving the key unlocked all day:

```
# ~/.gnupg/gpg-agent.conf
default-cache-ttl 3600
max-cache-ttl 28800
pinentry-program /opt/homebrew/bin/pinentry-mac
```

On Linux the `pinentry-program` line points at `pinentry-curses` or `pinentry-gnome3` instead. Reload with `gpg-connect-agent reloadagent /bye` after editing.

Tokens Held by Tools: the GitHub CLI

Some secrets are never typed by anyone. The GitHub CLI (`gh`) is the clearest example, and it illustrates both the right home for such a token and the way it can escape.

The Right Home

The fleet's project repositories use HTTPS remotes, and git obtains credentials for them from `gh`, which keeps its OAuth token in the macOS Keychain. The relevant git configuration is two lines:

```
[credential "https://github.com"]
  helper =
  helper = !gh auth git-credential
```

The empty first line clears any inherited helper so that only `gh` is asked. `gh auth status` confirms where the token lives; on a correctly configured Mac it reports the token as stored in the keyring. Two operational details follow from this arrangement:

- **Scopes matter.** Pushing a change to a GitHub Actions workflow file requires the `workflow` scope, which `gh` does not request by default. Without it, GitHub refuses the push with an explicit message, and `gh auth refresh -s workflow` adds the scope.
- **The Keychain is tied to the login session.** A plain SSH session into a Mac cannot read the Keychain, so `git pull` over HTTPS fails there with a request for a username. Pulls on a Mac are run from a local terminal.

How It Escaped

During the quarterly audit, two problems turned up with this token. On one Mac, `gh` had been storing it in a plain configuration file rather than the Keychain. On every machine, `~/.config/gh` was a symlink into the dotfiles repository, put there by an install script so that `gh` settings would be shared. The consequence was that `gh auth login` wrote the live token into a tracked directory, where it appeared as an uncommitted change on every machine. It was never committed, but a single `git add -A` would have published it.

The fix was an ignore rule for the token file, a fresh login that stored the token in the Keychain, and a general rule: before linking a tool's configuration directory into a repository, check what the tool writes there.

```
# gh writes its OAuth token here; never track it.
config/gh/hosts.yml
```

Cloud Credentials: Prefer Short-Lived Tokens

Long-lived cloud access keys are different in kind from API keys. They often authorize broad permissions, they are attractive to attackers, and their compromise can be expensive within minutes. The recommended direction is to stop holding them at all.

For AWS, IAM Identity Center (formerly AWS SSO) issues short-lived session tokens after a browser sign-in and caches them under `~/.aws/sso/cache/`, where they expire on their own:

```
# One-time profile setup (after the account-level configuration)
aws configure sso

# Each working session
aws sso login --profile research
aws sts get-caller-identity --profile research
```

These commands follow AWS's documentation and have not been exercised on the fleet for this post. Whatever form the credentials take, two rules from the placement table apply without exception: `~/.aws` is never synced or tracked, and it is included in the machine's encrypted backup so that a lost machine does not strand the configuration.

Rotation Discipline

Rotation is where secrets management stops being a filing exercise. The procedure below is the one that held up during the exposure, when a GPG private key, backup passphrases, cloud-drive OAuth tokens, SSH keys, and an API key had all been sitting in a synced folder for months and every one had to be treated as compromised.

When to Rotate

- **Exposure.** The secret reached a synced folder, a repository, a log, a chat or support transcript, a screen share, or a terminal that printed it. Exposure does not require evidence of misuse.
- **Expiry.** Keys with an expiry date (the GPG key in this series has a two-year expiry) are renewed or replaced before they lapse.
- **Change of trust.** A machine is retired, sold, or lost.

The Procedure

1. **Inventory.** List every place the secret is used: which machines, which tools, which scripts. This is where a `pass` hierarchy pays for itself.
2. **Issue the replacement first** where the service allows two live credentials, so that work does not stop.
3. **Update the store** (`pass edit` or `pass insert -f`) and push the store's git repository so every machine receives the new value.

4. **Revoke the old credential at the provider**, not merely delete it locally. Deleting a local copy revokes nothing.
5. **Test that the old credential is dead.** Call the service with it and confirm the refusal.
6. **Test that the new credential works** on every machine that uses it.
7. **Record what was rotated and when**, so the next audit can confirm it.

A Printed Secret Is an Exposed Secret

Twice during the audit a live token was printed in full while its configuration was being inspected: once a cloud-drive token shown by a configuration listing, and once the GitHub CLI token from the plain file described above. In both cases the output had gone to a terminal and a session transcript, and both tokens were rotated exactly as if they had been found in a public repository. The preventive habit is to inspect configuration with commands that redact, for example `gh auth status` rather than reading the token file, and to pipe listings through a filter before they reach the screen.

Test That the Old Credential Is Dead

A sweep of older credentials, made as part of the same audit, found most of them already dead. One API key issued more than a year earlier and long forgotten still worked, as did a code-assistant OAuth token found among the exposed settings. Both were revoked and confirmed dead. A credential's age says nothing about whether it is live; only a test does, and step 5 of the procedure exists because of this.

Revoking an OAuth Grant Logs Out Every Machine

Revoking the GitHub CLI's authorization on GitHub's website, intended to kill one printed token, logged out every machine at once. All of them had authorized the same OAuth application, and the revocation applied to the application grant, not to one machine's copy of the token. The cloud-drive provider behaved the same way. A revocation intended for one machine should therefore be planned as a revocation for all of them, with time set aside to log each one back in, and done before any unattended job depends on the token.

The Exposure as a Worked Example

For reference, the full rotation after the exposure covered:

- the GPG key behind the `pass` store, replaced and the store re-encrypted (details in Setting Up pass);
- the Borg backup passphrases (see Research Backup Architecture);
- the SSH keys used for the two code-hosting services;
- the `rclone` OAuth tokens for the cloud drives;
- the GitHub CLI token, a code-assistant token, and an email app password.

Every replacement went into `pass`, every old credential was revoked at its provider, and the tests in steps 5 and 6 were run for each.



Figure 3: A wooden pegboard with several distinct keys hanging in labeled silhouette outlines, one hook empty.

Catching Mistakes Before They Leave: Secret Scanning

Placement rules and careful habits reduce mistakes but do not eliminate them. The near-miss with the GitHub CLI token is exactly the case a pre-commit secret scanner exists to catch: a high-entropy token appearing in a staged file. The recommended practice is a scanner such as `gitleaks` run as a pre-commit hook in every repository that could plausibly receive a secret, starting with the dotfiles repository.

```
#!/bin/sh
# .git/hooks/pre-commit: refuse commits that contain likely secrets.
exec gitleaks protect --staged --redact
```

Recent `gitleaks` releases also accept `gitleaks git --staged` for the same check; consult the installed version's help. This hook is a recommendation and is not shown here as tested on the fleet. A scanner is a safety net, not a substitute for the placement rules: it cannot see a secret written to a synced folder, a log, or a terminal.

Verification

The following checks confirm the arrangement on one machine without printing any secret.

```
# No secrets exported from shell startup files
grep -nE 'export .*(KEY|TOKEN|SECRET)=' ~/.zshrc ~/.zprofile \
  ~/.zshenv 2>/dev/null

# Secret files are owner-only
ls -l ~/.Renviron ~/.env 2>/dev/null

# The dotfiles repository ignores them
git -C ~/dotfiles check-ignore -v config/gh/hosts.yml

# gh keeps its token in the keyring, with the expected scopes
gh auth status

# git asks gh for GitHub credentials
git config --global --get-regexp '^credential'
```

The first command should print nothing. The `ls` output should show `-rw-------` for each file, and `gh auth status` should report the keyring as the token's location.

Daily Workflow

Table 2: Everyday commands.

Task	Command	Notes
Add an API key	`pass insert api/service`	Prompts; nothing echoed
Generate a new secret	`pass generate api/service 32`	Generate first, then paste into the provider
Run a tool with a key	`with-secret VAR api/service cmd`	Visible only to `cmd`
Copy to clipboard	`pass -c api/service`	Clears after 45 seconds
List the inventory	`pass ls api`	Entry names only
Share a change	`pass git push`	Other machines run `pass git pull`
Check a tool login	`gh auth status`	Redacted; safe to display

Things to Watch Out For

1. **Clipboard managers defeat `pass -c`.** A clipboard history tool, or Universal Clipboard between two Macs, captures the secret before the 45-second clear. Exclude `pass` from clipboard history, or disable history on machines that handle secrets.
2. **Tools write tokens into their configuration directories.** If that directory is symlinked into a repository or a synced folder, the token goes with it. Check before linking.

3. **Diagnostics print the environment.** `env`, `printenv`, `set -x`, and verbose logging in scripts all reveal exported variables. This is the strongest practical argument for `with-secret` over `export`.
4. **Legacy exports survive a migration.** Moving keys into `pass` while leaving the old `export` lines in place leaves two copies. The first verification command finds them.
5. **Revocation is wider than it looks.** Revoking an OAuth application grant logs out every machine that uses it. Plan the re-login before revoking.
6. **The Keychain is unavailable over SSH.** Operations that need a Keychain-held token, such as an HTTPS `git pull` on a Mac, fail in a plain SSH session and should be run locally.
7. **Short-lived sessions expire mid-job.** A long script using short-lived cloud credentials can fail partway through when the session lapses; split long jobs or refresh before starting.



Figure 4: A GPG-style wax seal stamp resting beside a folded, sealed paper envelope on a dark desk.

What Did We Learn?

Lessons Learned

- **Placement is the control that matters most.** Every serious incident in the audit traced back to a secret sitting somewhere it should not have been, usually because a directory was linked into a synced folder or a repository for convenience.
- **Exposure is a state, not an event that needs proof.** Once a secret has been somewhere uncontrolled, the only safe response is rotation; trying to establish whether anyone read it is rarely possible and never conclusive.
- **Revocation must be verified.** Old credentials were found live, and only a direct test revealed it.

- **OAuth grants are shared across machines.** Treating a revocation as fleet-wide avoids an unplanned afternoon of logging back in.
- **Secrets need backup coverage like any other state.** A key that exists only on one machine is lost with that machine.

Limitations

- The arrangement is single-user. `pass` supports multiple recipients, but team secret sharing is not covered.
- It depends on GPG, whose key management is the hardest part of the whole scheme; Setting Up pass treats that in detail.
- The `.Renviron` compromise accepts a plaintext file on disk, protected by permissions and full-disk encryption rather than by GPG.
- Alternatives such as `gopass`, a self-hosted Bitwarden server, or the 1Password CLI are not evaluated here; `pass` was chosen for its small, auditable implementation.

Opportunities for Improvement

1. **Short-lived credentials wherever providers offer them**, starting with cloud access, so that a leaked credential expires on its own.
2. **A pre-commit secret scanner in every repository**, beginning with the dotfiles repository.
3. **Hardware-backed keys.** Moving the GPG key onto a hardware token makes every decryption require physical presence.
4. **A recovery script** that walks a fresh machine through importing the GPG key, cloning the store, and restoring tool logins, with checkpoints.
5. **Scheduled rotation for long-lived API keys**, rather than rotation only after exposure.

Wrapping Up

Secrets beyond passwords are easy to overlook because many of them are never typed: tools fetch and store them, and they sit quietly in configuration directories until something copies those directories somewhere else. The practices in this post reduce that risk to a small set of habits: decide where each class of secret lives, load secrets into one command at a time, keep tool tokens in the operating system's credential store, and rotate on exposure using a procedure that verifies the old credential is dead.

The main takeaways:

- Secrets never live in a synced folder or a dotfiles repository.
- Load secrets on demand; do not export them into the shell.
- A printed secret is an exposed secret, and exposure means rotation.
- After revoking, test that the old credential no longer works.
- Revoking an OAuth grant logs out every machine; plan for it.

See Also

Security series:

1. Security Foundations for a Multi-Machine Research Fleet
2. Setting Up pass: a Unix Password Manager
3. Secrets Management for the Workflow Construct (this post)
4. Migrating Off Dropbox: Beyond Dotfiles
5. Research Backup Architecture: Borg, Cloud Copies, and Time Machine
6. A Home Server for a Five-Machine Research Fleet
7. Object Storage for Research Data: The Layer Git Cannot Hold
8. Running the Quarterly Security Audit for the First Time: What Actually Broke

External references:

- `pass`: https://www.passwordstore.org/
- GnuPG: https://gnupg.org/
- GitHub CLI manual: https://cli.github.com/manual/
- AWS IAM Identity Center user guide: https://docs.aws.amazon.com/singlesignon/
- `gitleaks`: https://github.com/gitleaks/gitleaks

Reproducibility

Table 3: Versions in use.

Component	Version	Notes
Operating system	macOS 26.6	three Macs
Operating system	Linux Mint 22	Linux laptop and home server
Shell	zsh 5.9	
`pass`	1.7.4	
GnuPG	2.5	
GitHub CLI	2.101	Keychain token storage
AWS CLI	2.37	IAM Identity Center commands not exercised
`gitleaks`	not installed	recommended, untested here

Date of last verification: 2026-10-01 (`pass`, GnuPG, `gh`, and the verification commands, on macOS 26.6).

Let's Connect

Questions, suggestions, or corrections are welcome.

- **GitHub:** rgt47
- **Twitter/X:** @rgt47

- **LinkedIn:** Ronald Glenn Thomas
- **Email:** Contact form

I would like to hear from you if:

- You find an error, or a better approach to anything in this series.
- You run a small research fleet and have found where this design breaks.
- You want to compare notes on backups, secrets, or reproducible research infrastructure.